LABORATORY ASSIGNMENT

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 1
Student Name: B.Manoj
Roll No.: A24126552072
Date: 30-08-2026

---

1. TITLE

Implementation of Object-Oriented Inheritance Types in JavaScript (Single, Multilevel, and Hierarchical)

2. AIM

To design and implement Object-Oriented Programming (OOP) inheritance models in JavaScript using ES6 classes and the 'extends' keyword to demonstrate Single Inheritance, Multilevel Inheritance, and Hierarchical Inheritance.

3. OBJECTIVE

The objectives of this assignment are:

- To understand the core concepts of Object-Oriented Programming (OOP) in JavaScript ES6+.
- To implement Single Inheritance where a derived class inherits directly from a single parent class.
- To implement Multilevel Inheritance forming an inheritance chain across multiple class levels.
- To implement Hierarchical Inheritance where multiple derived classes inherit from a shared base class.
- To instantiate class objects and invoke both inherited and specialized member methods.

4. THEORY

Inheritance is a fundamental concept in Object-Oriented Programming that allows a derived class (child class) to inherit properties and methods from an existing base class (parent class), promoting code reusability and structured architecture.

In JavaScript ES6, inheritance is achieved using the 'class' and 'extends' keywords.

1. Single Inheritance: A single derived class inherits from one parent class (e.g., Bird -> Parrot).

2. Multilevel Inheritance: A class extends a derived class, creating an inheritance chain (e.g., Tree -> FruitTree -> MangoTree).

3. Hierarchical Inheritance: Multiple child classes inherit from the same parent base class (e.g., Instrument -> Guitar, Instrument -> Piano).

Application Flow:
Base Class Definition -> Child Class Extends Base -> Object Instantiation -> Method Execution -> Terminal Output

5. ALGORITHM / PROCEDURE

1. Create an assignments directory and create the subfolder inheritencetypes.

2. Create single.js: Define base class Bird with method fly(), extend Parrot from Bird with method speak(), instantiate Parrot and invoke fly() and speak().

3. Create multilevel.js: Define Tree class with grow(), extend FruitTree with fruit(), extend MangoTree with mango(), instantiate MangoTree and invoke grow(), fruit(), and mango().

4. Create hierarchical.js: Define Instrument class with play(), extend Guitar with strum(), extend Piano with press(), instantiate both classes and invoke their respective methods.

5. Execute each file in Node.js using 'node inheritencetypes/<filename>.js'.

6. Verify console outputs in the terminal against expected execution results.

---

6. PROGRAM

single.js

```
class Bird {
    fly() {
        console.log("Bird can fly");
    }
}

class Parrot extends Bird {
    speak() {
        console.log("Parrot can speak");
    }
}

let p = new Parrot();

p.fly();
p.speak();
```

multilevel.js

```
class Tree {
    grow() {
        console.log("Tree can grow");
    }
}

class FruitTree extends Tree {
    fruit() {
        console.log("Fruit tree gives fruit");
    }
}

class MangoTree extends FruitTree {
    mango() {
        console.log("Mango tree gives mangoes");
    }
}

let m = new MangoTree();

m.grow();
m.fruit();
m.mango();
```

hierarchical.js

```
class Instrument {
    play() {
        console.log("Instrument can play");
    }
}

class Guitar extends Instrument {
    strum() {
        console.log("Guitar is strumming");
    }
}

class Piano extends Instrument {
    press() {
        console.log("Piano is playing");
    }
}

let g = new Guitar();
let p = new Piano();

g.play();
g.strum();

p.play();
p.press();
```

7. CODE EXPLANATION

Code / Syntax	Explanation
class Parent { ... }	Defines the base class blueprint with shared member methods.
class Child extends Parent	Derives Child class from Parent, inheriting its properties and methods.
let p = new Parrot();	Instantiates an object of the derived class.
p.fly(); p.speak();	Invokes inherited parent method (fly) and child's own method (speak).
MangoTree extends FruitTree	Multilevel inheritance where MangoTree inherits from FruitTree which inherits from Tree.
Guitar extends Instrument	Hierarchical inheritance where multiple classes inherit from common Instrument class.

8. TEST CASES AND EXECUTION DATA

The assignment scripts were executed with the following test cases to verify functionality:

Test Case	Script / Input	Expected Output	Actual Output	Status
TC-01	node single.js	Bird can fly \n Parrot can speak	Bird can fly \n Parrot can speak	**Pass**
TC-02	node multilevel.js	Tree can grow \n Fruit tree gives fruit \n Mango tree gives mangoes	Tree can grow \n Fruit tree gives fruit \n Mango tree gives mangoes	**Pass**
TC-03	node hierarchical.js	Instrument can play \n Guitar is strumming \n Instrument can play \n Piano is playing	Instrument can play \n Guitar is strumming \n Instrument can play \n Piano is playing	**Pass**

9. OUTPUT

Output 1: Single Inheritance Output (single.js)

```
PS C:\Users\manoj\OneDrive\Desktop\fs_lab\assignments> node inheritencetypes/single.js
Bird can fly
Parrot can speak
```

Output 2: Multilevel Inheritance Output (multilevel.js)

```
PS C:\Users\manoj\OneDrive\Desktop\fs_lab\assignments> node inheritencetypes/multilevel.js
Tree can grow
Fruit tree gives fruit
Mango tree gives mangoes
```

Output 3: Hierarchical Inheritance Output (hierarchical.js)

```
PS C:\Users\manoj\OneDrive\Desktop\fs_lab\assignments> node inheritencetypes/hierarchical.j
Instrument can play
Guitar is strumming
Instrument can play
Piano is playing
```

10. RESULT

Thus, Single Inheritance, Multilevel Inheritance, and Hierarchical Inheritance were successfully implemented using JavaScript ES6 classes, executed in Node.js, and verified with accurate outputs across all test cases.